

第三十二讲：InnoDB Change Buffer

知春路遇上八里桥

<2026-09-01 Tue>



- 1 Change Buffer 引入
- 2 磁盘上的 Change Buffer
- 3 内存中的 Change Buffer
- 4 Change Buffer 写入路径 (缓冲)
- 5 Change Buffer 合并路径 (回放)
- 6 配置、实践与小结

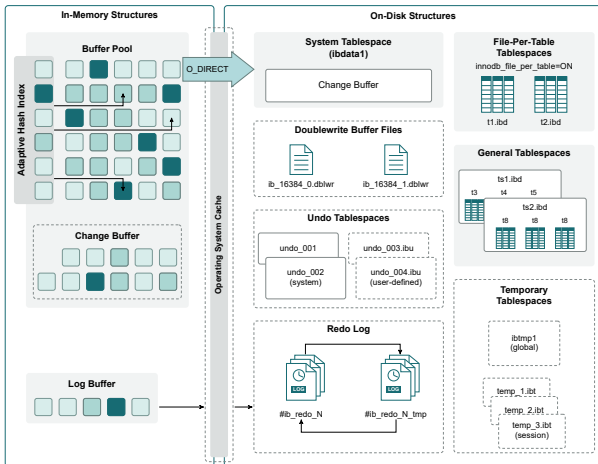


1

Change Buffer 引入



四个内存结构与本讲的位置

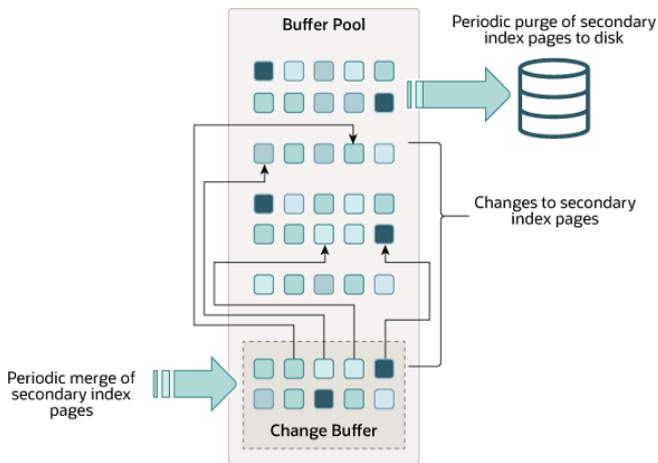


- 官方文档^a 将 InnoDB 的内存侧划分为四个结构
 - Buffer Pool:** 缓存数据页与索引页 —— 第三十、三十一讲
 - Change Buffer:** 二级索引页不在 Buffer Pool 时暂存其变更 —— 本讲
 - Adaptive Hash Index:** 为热点等值查询建立哈希捷径 —— 第三十三讲
 - Log Buffer:** mtr 提交时 redo 记录的落脚点 —— 第三十三讲
- 图中两处细节须先予辨明
 - Change Buffer 同时出现于 In-Memory 与系统表空间 ibdata1 —— 四者之中唯它跨越内存与磁盘
 - O_DIRECT 仅施加于数据文件与 dblwr 文件 (判据见 [os/os0file.cc](#) 第 3254 行); redo 文件不在其列, 故 Log Buffer 落盘须经 OS Cache 并显式 fsync (第三十三讲)

^a<https://dev.mysql.com/doc/refman/8.0/en/innodb-memory-structures.html>



官方文档中的 Change Buffer 图

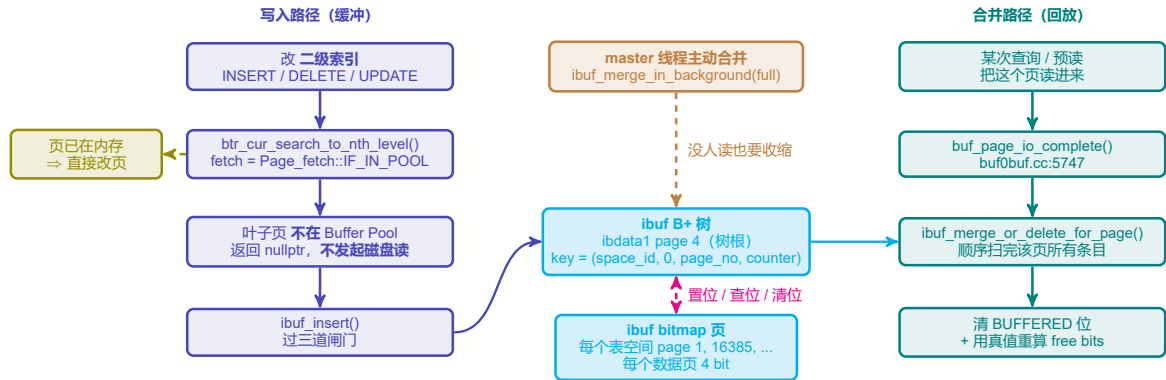


- 该图^a 表达三层含义
 - ① Change Buffer 物理上位于 Buffer Pool 之内，占用其一部分内存
 - ② Changes to secondary index pages: 二级索引页的变更先记入 Change Buffer，不直接施加于页
 - ③ Periodic merge / Periodic purge: 后台周期性地将条目合并回索引页，再由 page cleaner 刷盘
- 图中箭头均指向 Buffer Pool 内已驻留内存的页——合并以页已读入为前提

^a<https://dev.mysql.com/doc/refman/8.0/en/innodb-change-buffer.html>



全景图：缓冲与合并的两条路径



三道闸门 (任何一道不过就放弃缓冲, 退回“先读页再改页”):

- 1 目标页已在池中, 或页上有 pending purge (watch 已设) ⇒ 缓冲无意义 / 会乱序
- 2 记录大于空页可用空间的一半 ⇒ 几乎必然页分裂, 而“merge 必须成功”不允许调整树结构
- 3 水位: $size \geq max_size + 10$ 拒绝缓冲; 装不下 free bits 就置 do_merge 主动读页合并



Change Buffer 所要解决的问题

- 聚簇索引按主键顺序写入，磁盘侧基本为顺序 I/O；二级索引则不然
 - ▶ 二级索引键值分布随机，一次 INSERT 落于哪个叶子页无法预测
 - ▶ 若目标叶子页不在 Buffer Pool，为修改一条记录须先同步读入一个 16KB 页
 - ▶ 一条语句涉及多个二级索引时，随机读成倍放大
- 源码中的表述与文档一致：先记录变更，待页因其他操作被读入时再合并
 - ▶ 见 `storage/innobase/include/ibuf0ibuf.h` 的设计注释

```
78  /* The purpose of the insert buffer is to reduce random disk access.  
79  When we wish to insert a record into a non-unique secondary index and  
80  the B-tree leaf page where the record belongs to is not in the buffer  
81  pool, we insert the record into the insert buffer B-tree, indexed by  
82  (space_id, page_no). When the page is eventually read into the buffer  
83  pool, we look up the insert buffer B-tree for any modifications to the  
84  page, and apply these upon the completion of the read operation. This  
85  is called the insert buffer merge. */
```



Change Buffer 在四者中的特殊之处

- **唯一兼具内存与磁盘表示的结构**
 - ▶ 全部修改经由 mtr、写 redo，故崩溃可恢复，重启不丢
- **纯粹的性能优化**
 - ▶ 关闭后数据依然正确，仅二级索引写入退化为“先随机读页，再修改页”
 - ▶ 另三者之中，AHI 亦可关闭（只影响查询快慢），Log Buffer 不可关闭
- **合并异步进行，且跨越事务生命周期**
 - ▶ 官方文档明确指出：合并可能发生于事务提交之后，甚至服务器重启之后
 - ▶ 二级索引较多、涉及行数较大时，合并可能持续数小时并抬高磁盘 I/O
- 三者的横向对照置于第三十三讲末尾统一给出
- redo 的磁盘侧（LogFile / LogBlock / Checkpoint）见第二十八讲



预备知识: mini-transaction (mtr)

- 前已述及: **mtr** 保证一组页读写操作的原子性, 须显式开启与提交

```
mtr_start(&mtr); // 开启 MTR 事务  
mtr_commit(&mtr); // 提交 MTR 事务
```

- 一个 mtr 持有两个缓冲区, 见 [storage/innobase/include/mtr0mtr.h](#)

```
179 struct Impl {  
180     /** memo stack for locks etc. */  
181     mtr_buf_t m_memo;  
182  
183     /** mini-transaction log */  
184     mtr_buf_t m_log;  
185     ...  
186     /** specifies which operations should be logged; default  
187     value MTR_LOG_ALL */  
188     mtr_log_t m_log_mode;
```

- m_log: 本 mtr 产生的 redo 记录, 先积于私有缓冲, 不直接进入 Log Buffer
- m_memo: 本 mtr 持有的 latch 栈, 直至 commit 才统一释放
- 故 mtr 同时是两件事的原子单位: redo 记录的分组与 latch 的持有区间
- 注意 mtr 没有 rollback: 要么整组提交, 要么进程崩溃



预备知识: mtr 提交时的三个动作

- `mtr_t::commit()`, `storage/innobase/mtr/mtr0mtr.cc` 第 662 行

```
662 void mtr_t::commit() {  
663     ut_ad(is_active());  
664     m_impl.m_state = MTR_STATE_COMMITTING;  
665     Command cmd(this);  
666     if (has_any_log_record() ||  
667         (has_modifications() && m_impl.m_log_mode == MTR_LOG_NO_REDO)) {  
668         cmd.execute();  
669     } else {  
670         cmd.release_all();  
671         cmd.release_resources();  
672     }
```

- 实际执行者为 `mtr_t::Command::execute()` (第 842 行), 其顺序为
 - ① 将 `m_log` 中的 redo 整段拷入 Log Buffer (第三十三讲展开)
 - ② 将本 mtr 修改过的脏页挂入 `flush_list`
 - ③ `release_all()` 释放 `m_memo` 中全部 latch
- 单条 redo 记录置 `MLOG_SINGLE_REC_FLAG`, 多条则以 `MLOG_MULTI_REC_END` 收尾 —— 崩溃恢复据此判定该组是否完整 (第三十四讲)
- 本讲何以在意: 位图与页修改均经由 mtr, “是否同一 mtr” 直接决定崩溃恢复能否观察到中间状态



Change Buffer 的观测

- SHOW ENGINE INNODB STATUS 的段落拼装见 [storage/innobase/srv/srv0srv.cc](#) 第 1441 行，数字由 `ibuf_print()` 输出（见 [ibuf/ibuf0ibuf.cc](#) 第 4381 行）

```
mysql> show engine innodb status\G
-----
INSERT BUFFER AND ADAPTIVE HASH INDEX
-----
Ibuf: size 1, free list len 0, seg size 2, 0 merges
merged operations:
  insert 0, delete mark 0, delete 0
discarded operations:
  insert 0, delete mark 0, delete 0
```

- size / free list len / seg size 直接取自控制结构 `ibuf_t` 的同名字段（该结构见第三节）
- merges 为已合并的页数，其下两行是按操作类型划分的条目计数
- discarded 非零表示存在因表空间被删或索引被 drop 而直接丢弃的条目
- 空库上 seg size 2 即“一个 header 页 + 一个空的树根页”，扣除 header 后 size 1 —— 这两个页的固定位置见下一节



2

磁盘上的 Change Buffer



一棵全局的树 + 每个表空间一份位图



位置固定: `ibuf_bitmap_page()` 判定 `page_no & (page_size - 1) == 1` $\Rightarrow$ page 1, 16385, 32769 ...

注意 每个表空间都有一份位图, 但 **ibuf B+ 树全局只有一棵** (在 space 0)

每个数据页占 `IBUF_BITS_PER_PAGE = 4 bit` $\Rightarrow 16384 \times 4 \text{ bit} = 8\text{KB}$, 正好放进一页

IBUF_BITMAP_FREE bit 0..1	BUFFERED bit 2	IBUF bit 3
------------------------------	-------------------	---------------

FREE: 剩余空间的 **粗粒度**档位, 量化单位 = `page_size/32` (16KB 页 = 512B)
只有 4 档: 0 / 512 / 1024 / 2048 字节, 且 `n==3` 被 **硬降为 2**

BUFFERED: 有没有待合并的变更——读页时的快速开关, 决定要不要去 `ibuf` 树查一趟

IBUF: 这页是不是 `ibuf` 树自己的页 (或在它的 `free list` 里)

不变式: merge 必须成功 $\Rightarrow$ free bits **绝不能高于**页上真实空闲空间; 只做 **保守低估**: 宁可少缓冲, 也不能“超卖”。
减小 / 清零可以单独提交一个 `mtr`; **增大必须与页修改同 mtr**——否则崩溃恢复中途 free bits 会一时偏高



代码: 树与位图页的固定位置

① IBUF_INDEX 树索引页, 位于系统表空间 ibdata1 (SPACE_ID=0)

- 见 `storage/innobase/include/ibuf0types.h`

```
37  /* The insert buffer tree itself is always located in space 0. */
38  #define IBUF_SPACE_ID static_cast<space_id_t>(0)
```

- 两个固定页号, 见 `storage/innobase/include/fsp0types.h`

```
156  /** insert buffer bitmap; The ibuf bitmap pages are the ones whose page number
157  is the number above plus a multiple of XDES_DESCRIBED_PER_PAGE */
158  constexpr uint32_t FSP_IBUF_BITMAP_OFFSET = 1;
159  /** in every tablespace */
160  constexpr uint32_t FSP_FIRST_INODE_PAGE_NO = 2;
161  /** The following pages exist in the system tablespace (space 0). */
162  /** insert buffer header page, in tablespace 0 */
163  constexpr uint32_t FSP_IBUF_HEADER_PAGE_NO = 3;
164  /** insert buffer B-tree root page in tablespace 0;
165  The ibuf tree root page number in tablespace 0; its fseg inode is on the page
166  number FSP_FIRST_INODE_PAGE_NO */
167  constexpr uint32_t FSP_IBUF_TREE_ROOT_PAGE_NO = 4;
```

- header 位于 ibdata1 的 page 3、B+ 树根位于 page 4, 树的 fseg inode 在 page 2

② IBUF_BITMAP: 位图页, 每个独立表空间 (ibd) 内部均有

- 位置判定 `ibuf_bitmap_page()`, 见 `include/ibuf0ibuf.ic` 第 147 行
- 每页 4 bit 的常量 `IBUF_BITS_PER_PAGE`, 见 `ibuf/ibuf0ibuf.cc` 第 49 行



代码: 位图中 4 个 bit 的定义

- 见  storage/innobase/ibuf/ibuf0ibuf.cc

```
240  /** @name Offsets to the per-page bits in the insert buffer bitmap */
241  /** @{ */
242  /** Bits indicating the amount of free space */
243  constexpr uint32_t IBUF_BITMAP_FREE = 0;
244  /** true if there are buffered changes for the page */
245  constexpr uint32_t IBUF_BITMAP_BUFFERED = 2;
246  /** true if page is a part of the ibuf tree, excluding the root page, or is in
247  the free list of the ibuf */
248  constexpr uint32_t IBUF_BITMAP_IBUF = 3;
249  /** @} */
```

- 三个常量即 bit 编号: FREE 自 bit 0 起占两位, BUFFERED 与 IBUF 各占一位
- 位图中不存在第四个字段, 其余语义全部由 FREE 这两位承载



代码: free bits 的四个档位

- 见  storage/innobase/include/ibuf0ibuf.ic

```
44  constexpr uint32_t IBUF_PAGE_SIZE_PER_FREE_SPACE = 32;

158 static inline uint ibuf_index_page_calc_free_bits(uint page_size,
159                                                    uint max_ins_size) {
160     uint n = max_ins_size / (page_size / IBUF_PAGE_SIZE_PER_FREE_SPACE);
161     if (n == 3) { n = 2; }           // 3 档并入 2 档
162     if (n > 3) { n = 3; }           // 上限 3
163     return n;
164 }
```

- 反向换算为 `ibuf_index_page_calc_free_from_bits()` (第 181 行), `bits=3` 时返回 $4 * 512 = 2048$ —— 写入与读出两侧均按 2048 计, 不会将 `bits=3` 当作第四档的上界
- 此处两个 `if` 即下一页那条不变式在代码中的落点



不变式的出处: merge must always succeed

- 见  storage/innobase/include/ibuf0ibuf.h , 紧接第 78 行的设计注释之后

```
87  /* The insert buffer merge must always succeed. To guarantee this,  
88  the insert buffer subsystem keeps track of the free space in pages for  
89  which it can buffer operations. Two bits per page in the insert  
90  buffer bitmap indicate the available space in coarse increments. The  
91  free bits in the insert buffer bitmap must never exceed the free space  
92  on a page. It is safe to decrement or reset the bits in the bitmap in  
93  a mini-transaction that is committed before the mini-transaction that  
94  affects the free space. It is unsafe to increment the bits in a  
95  separately committed mini-transaction, because in crash recovery, the  
96  free bits could momentarily be set too high. */
```

- 缓冲意味着延迟而非放弃: 条目一旦进入 ibuf 树, 合并时就必须容纳得下
- 末句正是前述 mtr 的用处: 单独提交意味着崩溃恢复可能只重放至位图那一步, 此时 free bits 偏小无害, 偏大则使后续合并放不下
- 该不变式解释了后文几乎所有“偏于保守”的设计



图解：一条 INSERT 在 ibuf 中的形态

● 例: `t(id INT PK, age INT, name VARCHAR(32), KEY idx_age(age))`, 执行 `INSERT INTO t VALUES (1000, 27, 'abc')`

目标叶子页: 不在 Buffer Pool
t.ibd space_id = 42 / page_no = 57 / 16KB

idx_age 的二级索引记录 (age, id) = (27, 1000)
只有索引列 + 主键列, name 列一个字节都不进来

定位: 哪个表空间的哪个页

元信息

数据: 索引记录原样照抄

SPACE 4B = 42 00 00 00 2a	MARK 1B = 0 00	PAGE 4B = 57 00 00 00 39	METADATA 4 + 2 × 6 = 16B 展开见下方	USER+0 age 4B = 27 80 00 00 1b	USER+1 id 4B = 1000 80 00 03 e8
---	--------------------------------	--	---	--	---

整条约 45B
33B 数据 + 旧格式头 6B
+ 6 个字段偏移 6B
省掉一次 16384B
的同步随机读

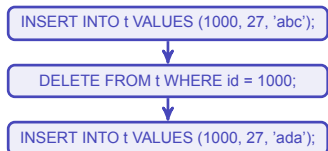
COUNTER 2B = 0 本页第一条 00 00	TYPE 1B IBUF_OP_INSERT 三种 op 只差这 1 字节 00	FLAGS 1B IBUF_REC_COMPACT 行格式标记 01	age 的类型 6B INT / len 4 可空 06 03 00 04 00 00	id 的类型 6B INT / len 4 NOT NULL 06 03 00 04 80 00
--	--	--	--	---

每列 6 字节 = DATA_NEW_ORDER_NULL_TYPE_BUF_SIZE: [0] mtype [1] MySQL 类型码 [2..3] 长度 [4..5] 字符集号 (最高位借做 NOT NULL) ——类型自带是因为合并时表定义可能已经没了, 靠 `ibuf_dummy_index_create()` 造个临时 index 解析



图解：同一页上的三条变更与回放顺序

□ 用户依次执行三条 SQL



期间 $space=42 / page=57$ 一直 **不在 Buffer Pool**，所以三次改动都只落进 ibuf 树

□ 页终于被读进来，按 **counter** 顺序逐条回放

□ ibuf 树叶子上三条物理相邻的记录

前缀完全相同		只有 counter + op 这 3 字节不同		用户列一样
42 0 57	0	INSERT	27, 1000	
42 0 57	1	DEL MARK	27, 1000	
42 0 57	2	INSERT	27, 1000	

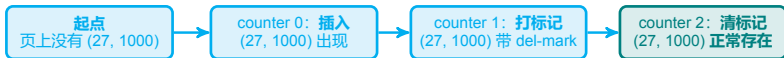
ibuf **不做抵消**：“插了又删”不会消掉条目。
COUNTER 由 `ibuf_get_entry_counter()` 现算：
PAGE_CUR_LE 找到该 (space, page) 的最后一条，加一

为什么必须有 counter

排序键前缀一模一样，**没有 counter 就没有稳定顺序**——回放可能停在“已打删除标记”，一条本该可见的记录凭空消失

旧格式没有 counter

所以 `ibuf_insert()` 遇到旧式记录**直接拒绝缓冲**，让老条目先合并掉，逐步换成新格式



第三条不是“再插一遍”：`ibuf_insert_to_index_page()` 发现 `low_match == n_fields`，走 `btr_cur_set_deleted_flag_for_ibuf(rec, ..., false)` **原地清掉删除标记**
(两次 INSERT 的 name 不同，但 name 不在 `idx_age` 里，索引记录一模一样)



代码: ibuf 记录的字段划分

- 见 `storage/innobase/ibuf/ibuf0ibuf.cc`

```
251  /** in the pre-4.1 format, the page number. later, the space_id */
252  constexpr uint32_t IBUF_REC_FIELD_SPACE = 0;
253  /** starting with 4.1, a marker consisting of 1 byte that is 0 */
254  constexpr uint32_t IBUF_REC_FIELD_MARKER = 1;
255  /** starting with 4.1, the page number */
256  constexpr uint32_t IBUF_REC_FIELD_PAGE = 2;
257  /** the metadata field */
258  constexpr uint32_t IBUF_REC_FIELD_METADATA = 3;
259  /** first user field */
260  constexpr uint32_t IBUF_REC_FIELD_USER = 4;
```

- MARKER 恒为 0，用以区分 4.1 之前的旧格式（旧格式首字段即 page_no）
- 前三个字段构成排序前缀，合并时一次 PAGE_CUR_GE 定位即可顺序扫完该页的条目



代码: METADATA 字段

- 见 `storage/innobase/ibuf/ibuf0ibuf.cc` (精简代码, IBUF_REC_COMPACT 实际位于第 285 行)

```
269  /** Combined size of info fields at the beginning of the fourth field */
270  constexpr uint32_t IBUF_REC_INFO_SIZE = 4;

274  /* Offsets for the fields at the beginning of the fourth field */
275  /** Operation counter */
276  constexpr uint32_t IBUF_REC_OFFSET_COUNTER = 0;
277  /** Type of operation */
278  constexpr uint32_t IBUF_REC_OFFSET_TYPE = 2;
279  /** Additional flags */
280  constexpr uint32_t IBUF_REC_OFFSET_FLAGS = 3;
281  /** Set in IBUF_REC_OFFSET_FLAGS if the user index is in COMPACT format */
282  constexpr uint32_t IBUF_REC_COMPACT = 0x1;
```

- 拼装于 `ibuf_entry_build()` (第 1526 行), 解析于 `ibuf_build_entry_from_ibuf_rec_func()` (第 1335 行, inline 包装在第 1403 行)
- IBUF_REC_INFO_SIZE = 4 仅计 counter / type / flags 这 4 字节, 其后按列数追加每列的类型信息



三个易混淆的“不存”

- **不存页**：既非 16KB 页的副本，亦非页的差分补丁 —— Change Buffer 中没有页，只有“对某个页待施加的操作”
- **不存整行**：仅存该一个二级索引的记录（索引列...，主键列...）
- **不存事务信息**：二级索引记录本身即无 `trx_id / roll_ptr`，ibuf 记录亦然；可见性判定须回聚簇索引，与 ibuf 无关
- 三种 op 的 entry 用途不同：INSERT 的 entry 是待插入的记录本身，DELETE_MARK / DELETE 的 entry 仅用于在页上定位目标记录
- 由此可解释一处代码：`ibuf_rec_get_volume_func()`（第 1454 行）对后两种直接 `return 0`（判据在第 1480 行）—— 打标记不占空间，物理删除是腾出空间，但一律保守按 0 计



3

内存中的 Change Buffer



核心控制结构 ibuf_t

- 全局单例 ibuf, 见 `storage/innobase/include/ibuf0ibuf.ic`

```
62 struct ibuf_t {
63     ulint size;           /*!< current size of the ibuf index tree, in pages */
64     ulint max_size;       /*!< recommended maximum size of the ibuf index tree,
65                            in pages */
66     ulint seg_size;       /*!< allocated pages of the file segment containing
67                            ibuf header and tree */
68     bool empty;           /*!< Protected by the page latch of the root page of
69                            the insert buffer tree
70                            (FSP_IBUF_TREE_ROOT_PAGE_NO). true if and only if the
71                            insert buffer tree is empty. */
72     ulint free_list_len;  /*!< length of the free list */
73     ulint height;         /*!< tree height */
74     dict_index_t *index; /*!< insert buffer index */
75
76     std::atomic<ulint> n_merges; /*!< number of pages merged */
77     std::atomic<ulint> n_merged_ops[IBUF_OP_COUNT];
78     /*!< number of operations of each type merged to index pages */
79     std::atomic<ulint> n_discarded_ops[IBUF_OP_COUNT];
80     /*!< number of operations of each type discarded without merging due to
81        the tablespace being deleted or the index being dropped */
82 };
```

- index 是一个内存中伪造的 dict_index_t, 构造于 ibuf_init_at_db_start((第 458 行), 它并不存在于数据字典之中



size / max_size / seg_size 的来历

- size 与 free_list_len 于每次改动树后由 ibuf_size_update() 重算

▶ 见 `storage/innobase/ibuf/ibuf0ibuf.cc`

```
443 static void ibuf_size_update(const page_t *root) {
444     ut_ad(mutex_own(&ibuf_mutex));
445     ibuf->free_list_len =
446         flst_get_len(root + PAGE_HEADER + PAGE_BTR_IBUF_FREE_LIST);
447     ibuf->height = 1 + btr_page_get_level(root);
448     /* the '1 +' is the ibuf header page */
449     ibuf->size = ibuf->seg_size - (1 + ibuf->free_list_len);
450 }
```

- max_size 由 ibuf_max_size_update() 按 Buffer Pool 的百分比换算为页数

▶ 见 `storage/innobase/ibuf/ibuf0ibuf.cc`

```
531 void ibuf_max_size_update(ulint new_val) {
532     ulint new_size =
533         ((buf_pool_get_curr_size() / UNIV_PAGE_SIZE) * new_val) / 100;
534     mutex_enter(&ibuf_mutex);
535     ibuf->max_size = new_size;
536     mutex_exit(&ibuf_mutex);
537 }
```

- ibuf 树的 free list 挂于根页头部 (PAGE_BTR_IBUF_FREE_LIST), 预留页存于此处, 以免插入时临时向 fsp 申请页而破坏 latch 顺序



三把互斥量与三条水位线

- 见 `storage/innobase/ibuf/ibuf0ibuf.cc`

```
286  /** The mutex used to block pessimistic inserts to ibuf trees */
287  static ib_mutex_t ibuf_pessimistic_insert_mutex;
288  /** The mutex protecting the insert buffer structs */
289  static ib_mutex_t ibuf_mutex;
290  /** The mutex protecting the insert buffer bitmaps */
291  static ib_mutex_t ibuf_bitmap_mutex;

310  const uint IBUF_CONTRACT_ON_INSERT_NON_SYNC = 0;    // 310: 超出即异步收缩
311  const uint IBUF_CONTRACT_ON_INSERT_SYNC      = 5;    // 315: 超出 5 页转同步
312  const uint IBUF_CONTRACT_DO_NOT_INSERT       = 10;   // 320: 超出 10 页拒绝缓冲
```

- 三条水位均以 `ibuf->max_size` 为基准，逐级加压

- ① `size >= max_size + 0`: 插入成功后附带一次异步收缩
- ② `size >= max_size + 5`: 收缩转为同步（等待读 I/O 完成）
- ③ `size >= max_size + 10`: 先同步收缩，随后放弃缓冲，回退为读页

- `ibuf->size / max_size` 在多处是有意的脏读：注释说明最坏情形仅是多做或少做一次收缩，不影响正确性



4

Change Buffer 写入路径 (缓冲)



入口: B+ 树下降时发现叶子页不在内存

- 起点为 `btr_cur_search_to_nth_level()`, 文件 `storage/innobase/btr/btr0cur.cc`

```
942 } else if (latch_mode <= BTR_MODIFY_LEAF) {
943     rw_latch = latch_mode;
944
945     if (btr_op != BTR_NO_OP &&
946         ibuf_should_try(index, btr_op != BTR_INSERT_OP)) {
947         /* Try to buffer the operation if the leaf
948          * page is not in the buffer pool. */
949         fetch = btr_op == BTR_DELETE_OP ? Page_fetch::IF_IN_POOL_OR_WATCH
950             : Page_fetch::IF_IN_POOL;
951     }
952 }
```

- BTR_DELETE_OP 采用 IF_IN_POOL_OR_WATCH: purge 一并设置 watch
- 取得 nullptr 后按 btr_op 分派至 `ibuf_insert()` (第 978、990、1006 行)
 - ▶ 三种 cursor 结果: BTR_CUR_INSERT_TO_IBUF / BTR_CUR_DEL_MARK_IBUF / BTR_CUR_DELETE_IBUF
 - ▶ 缓冲失败则改回正常 fetch 模式 goto `retry_page_get`, 转为读页

预筛: ibuf_should_try()

- 见 `storage/innobase/include/ibuf0ibuf.ic`

```
116 static inline bool ibuf_should_try(  
117     dict_index_t *index,      /*!< in: index where to insert */  
118     ulint ignore_sec_unique) /*!< in: if != 0, we should ignore UNIQUE  
119                             constraint on a secondary index when we decide */  
120 {  
121     return (innodb_change_buffering != IBUF_USE_NONE && ibuf->max_size != 0 &&  
122             index->space != dict_sys_t::s_dict_space_id &&  
123             !index->is_clustered() && !dict_index_is_spatial(index) &&  
124             !dict_index_has_desc(index) &&  
125             index->table->quiesce == QUIESCE_NONE &&  
126             (ignore_sec_unique || !dict_index_is_unique(index)) &&  
127             srv_force_recovery < SRV_FORCE_NO_IBUF_MERGE);  
128 }
```

- 逐条对应：功能未关闭；ibuf 有容量；非数据字典表空间；非聚簇索引；非空间索引；无降序列；表不处于 quiesce；非唯一索引；force_recovery 未禁用 ibuf merge
- 排除唯一索引的理由：插入前必须确认无重复键，此判定绕不开读页，缓冲遂无意义
 - 但 BTR_INSERT_OP 以外的操作（del mark / delete）传入 ignore_sec_unique = true，因其无需唯一性校验



三种可缓冲的操作与总开关

- 见 `storage/innobase/include/ibuf0ibuf.h`，注意这些值会写到磁盘上

```
50  /* Possible operations buffered in the insert/whatever buffer. See
51  ibuf_insert(). DO NOT CHANGE THE VALUES OF THESE, THEY ARE STORED ON DISK. */
52  typedef enum {
53      IBUF_OP_INSERT = 0,
54      IBUF_OP_DELETE_MARK = 1,
55      IBUF_OP_DELETE = 2,
56      /* Number of different operation types. */
57      IBUF_OP_COUNT = 3
58  } ibuf_op_t;
59  /** Combinations of operations that can be buffered.
60  @see innodb_change_buffering_names */
61  enum ibuf_use_t {
62      IBUF_USE_NONE = 0,
63      IBUF_USE_INSERT,           /* insert */
64      IBUF_USE_DELETE_MARK,     /* delete */
65      IBUF_USE_INSERT_DELETE_MARK, /* insert+delete */
66      IBUF_USE_DELETE,         /* delete+purge */
67      IBUF_USE_ALL              /* insert+delete+purge */
68  };
```

- INSERT 源于 INSERT; DELETE_MARK 源于 DELETE / UPDATE 的删除标记; DELETE 源于 purge 的物理删除
- ibuf_use_t 与 innodb_change_buffering 的 6 个取值一一对应 (名字表见 `handler/ha_innodb.cc` 第 573 行):

▶ none / inserts / deletes / changes / purges / all, 默认 all



ibuf_insert(): 以 $op \times use$ 作二维分派

- 见  storage/innobase/ibuf/ibuf0ibuf.cc 第 3282 行 (精简代码)

```
3290  ibuf_use_t use = static_cast<ibuf_use_t>(innodb_change_buffering);
3291  ut_a(!index->is_clustered());           // 3300 行
3292  auto no_counter = use <= IBUF_USE_INSERT; // 3302 行
3293
3294  switch (op) {                             // 3304 行
3295      case IBUF_OP_INSERT:                  // none/delete/delete_mark -> 拒绝
3296          ... goto check_watch;
3297      case IBUF_OP_DELETE_MARK:             // none/insert -> 拒绝
3298          ... goto check_watch;
3299      case IBUF_OP_DELETE:                 // none/insert/insert+delete -> 拒绝
3300          ... goto skip_watch;             // 注意跳过 watch 检查
3301  }
```

- no_counter: 仅缓冲 insert 时, 同一页内多条 insert 无需保序, 可省去 counter
- IBUF_OP_DELETE 走 skip_watch 是一处关键细节: purge 自身已先调 buf_pool_watch_set(), 若再检查 watch 则 purge 永远无法被缓冲

闸门一：目标页已在 Buffer Pool 或已被 watch

- 见 `storage/innobase/ibuf/ibuf0ibuf.cc` (精简代码)

```
3350 check_watch:
3351     /* If a thread attempts to buffer an insert on a page while a
3352     purge is in progress on the same page, the purge must not be
3353     buffered, because it could remove a record that was
3354     re-inserted later. For simplicity, we block the buffering of
3355     all operations on a page that has a purge pending. */
3356     {
3357         buf_pool_t *buf_pool = buf_pool_get(page_id);
3358         buf_page_t *bpage = buf_page_get_also_watch(buf_pool, page_id);
3359         if (bpage != nullptr) {
3360             /* A buffer pool watch has been set or the
3361             page has been read into the buffer pool.
3362             Do not buffer the request. */
3363             return false;
3364         }
3365     }
```

- `buf_page_get_also_watch()` 将两种情形归为一次判定：页已被读入（含并发正在读入），或 watch 已设置
- 注意此处仅是窥视，既不读入页，也不对页加 latch



闸门二：记录过大

- 承接 skip_watch 标签，见 [storage/innobase/ibuf/ibuf0ibuf.cc](#)

```
3378 skip_watch:
3379     entry_size = rec_get_converted_size(index, entry);
3380
3381     if (entry_size >=
3382         page_get_free_space_of_empty(dict_table_is_comp(index->table)) / 2) {
3383         return false;
3384     }
```

- 阈值取 `page_get_free_space_of_empty()` 的一半，即空页的可用空间，与目标页当前的充盈程度无关 —— 无需读页即可计算
- 二者通过后才进入真正的插入 `ibuf_insert_low()`，第三道闸门（水位与空间核算）位于其内



闸门三 (上): ibuf_insert_low() 的水位检查与页预留

- 见  `storage/innobase/ibuf/ibuf0ibuf.cc` 第 2977 行 (精简代码)

```
3015 if (ibuf->max_size == 0) { return DB_STRONG_FAIL; }
3016
3017 if (ibuf->size >= ibuf->max_size + IBUF_CONTRACT_DO_NOT_INSERT) {
3018     /* Insert buffer is now too big, contract it but do not try to insert */
3019     ibuf_contract(true);
3020     return DB_STRONG_FAIL;          // 交给调用方去读页
3021 }
3022
3023 ibuf_entry = ibuf_entry_build(op, index, entry, page_id.space(),
3024                               page_id.page_no(),
3025                               no_counter ? ULINT_UNDEFINED : 0xFFFF, heap);
3026
3027 if (BTR_LATCH_MODE_WITHOUT_INTENTION(mode) == BTR_MODIFY_TREE) {
3028     for (;;) {                      // 悲观模式: 先备好空闲页
3029         mutex_enter(&ibuf_pessimistic_insert_mutex);
3030         mutex_enter(&ibuf_mutex);
3031         if (UNIV_LIKELY(ibuf_data_enough_free_for_insert())) { break; }
3032         mutex_exit(&ibuf_mutex);
3033         mutex_exit(&ibuf_pessimistic_insert_mutex);
3034         if (!ibuf_add_free_page()) { return DB_STRONG_FAIL; }
3035     }
3036 }
```

- 第三道闸门的前半段即前述最高水位: `size >= max_size + 10` 时收缩一次并返回 `DB_STRONG_FAIL`, 令调用方转为读页
- 构造 entry 时 counter 先填 `0xFFFF`, 以便用 `PAGE_CUR_LE` 定位到该 (space, page) 的最后一条记录, 再读出真实 counter 并加一



闸门三 (下): ibuf_insert_low() 的空间核算与置位

- 先将闸门一复查一次 (第 3129 行): 取得位图页 latch 期间页可能已被读入, 另外页上存在显式行锁时亦不缓冲 `if (buf_page_peek(page_id) || lock_rec_expl_exist_on_page(page_id))`
- 定位游标后先统计该页已缓冲的总量, 再与位图中的 free bits 比较

```
3076 min_n_recs = 0;
3077 buffered = ibuf_get_volume_buffered(&pcur, page_id.space(), page_id.page_no(),
3078                                     op == IBUF_OP_DELETE ? &min_n_recs
3079                                     : nullptr, &mtr);
3080 if (op == IBUF_OP_DELETE &&
3081     (min_n_recs < 2 || buf_pool_watch_occurred(page_id))) {
3082     /* The page could become empty after the record is deleted, or the
3083     page has been read in to the buffer pool. Refuse to buffer. */
3084     ...
3085 }
3134 if (op == IBUF_OP_INSERT) {
3135     ulint bits = ibuf_bitmap_page_get_bits(bitmap_page, page_id, page_size,
3136                                           IBUF_BITMAP_FREE, &bitmap_mtr);
3137     if (buffered + entry_size + page_dir_calc_reserved_space(1) >
3138         ibuf_index_page_calc_free_from_bits(page_size, bits)) {
3139         do_merge = true; // 装不下, 改为触发合并
3140         ibuf_get_merge_page_nos(false, pcur.get_rec(), &mtr, space_ids,
3141                                page_nos, &n_stored);
3142         goto fail_exit;
3143     }
3144 }
```

- `min_n_recs < 2` 时才拒绝 delete: 页可能被删空, 而 B+ 树不允许非根空页
- 容纳不下则置 `do_merge = true`, 出函数时调用 `buf_read_ibuf_merge_pages()` 主动读页合并

乐观插入 ⇒ 悲观插入

- ibuf 树本身亦是 B+ 树，插入同样分乐观 / 悲观两种模式

```
3175 old_bit_value = ibuf_bitmap_page_get_bits(bitmap_page, page_id, page_size,
3176                                             IBUF_BITMAP_BUFFERED, &bitmap_mtr);
3177 if (!old_bit_value) {
3178     ibuf_bitmap_page_set_bits(bitmap_page, page_id, page_size,
3179                               IBUF_BITMAP_BUFFERED, true, &bitmap_mtr);
3180 }
3386 err = ibuf_insert_low(BTR_MODIFY_PREV, op, no_counter, entry, entry_size,
3387                      index, page_id, page_size, thr);
3388 if (err == DB_FAIL) {
3389     err = ibuf_insert_low(BTR_MODIFY_TREE | BTR_LATCH_FOR_INSERT, op,
3390                          no_counter, entry, entry_size, index, page_id,
3391                          page_size, thr);
3392 }
```

- BUFFERED 位为幂等置位：仅在自 0 变 1 时写入，省去重复的位图 redo
- 悲观插入成功后调用 `ibuf_contract_after_insert(entry_size)`（第 2449 行）：按刚插入的字节数等量收缩，超出 +5 页即转为同步



5

Change Buffer 合并路径 (回放)



统一入口 ibuf_merge_or_delete_for_page()

- 一个函数承担两种语义：**apply**（合并回页）与 **delete**（直接丢弃条目）
 - `ibuf/ibuf0ibuf.cc` 第 3964 行，四个调用点：

调用点	位置	语义
<code>buf_page_io_complete()</code>	▸ <code>buf0buf.cc</code> 第 5747 行	apply
<code>Buf_fetch<T>::zip_page_handler()</code>	▸ <code>buf0buf.cc</code> 第 3953 行	apply
<code>buf_page_create()</code>	▸ <code>buf0buf.cc</code> 第 5112 行	delete
<code>buf_read_ibuf_merge_pages()</code>	▸ <code>buf0rea.cc</code> 第 616、637 行	delete

- 两种语义由首参 `block` 区分：非空则合并回该页，为 `nullptr` 则仅丢弃条目
- `buf_page_io_complete()` 是主路径：读 I/O 完成、页尚处 `io-fix` 状态时合并，条件为索引叶子页、非临时/undo 表空间、非 `stale` 页
- `buf_page_create()` 属清理：新建页时清除残留条目（原索引 `drop` 后的遗留）
- `buf_read_ibuf_merge_pages()` 中的两处为表空间已删除时的丢弃分支



快速跳过与合并主循环

- 进入函数先查位图，无 BUFFERED 位即返回（此为最常见路径）

```
4030 bitmap_bits = ibuf_bitmap_page_get_bits(bitmap_page, page_id, *page_size,  
4031                                         IBUF_BITMAP_BUFFERED, &mtr);  
4032 ibuf_mtr_commit(&mtr);  
4033 if (!bitmap_bits) {  
4034     /* No inserts buffered for this page */  
4035     fil_space_release(space);  
4036     return;  
4037 }
```

- 其后由 `ibuf_search_tuple_build()` 构造 `(space, page_no)` 前缀，`PAGE_CUR_GE` 定位后自 loop:（第 4084 行）顺序扫描，逐条按 op 回放

```
4154 switch (op) {  
4155     case IBUF_OP_INSERT:  
4156         ibuf_insert_to_index_page(entry, block, dummy_index, &mtr);    break;  
4157     case IBUF_OP_DELETE_MARK:  
4158         ibuf_set_del_mark(entry, block, dummy_index, &mtr);           break;  
4159     case IBUF_OP_DELETE:  
4160         ibuf_delete(entry, block, dummy_index, &mtr);                   break;  
4161 }
```

- `IBUF_OP_DELETE` 须 latch 位图页，为守 latch 顺序必须先提交 mtr、保存游标位置，再由 `ibuf_restore_pos()` 恢复



收尾: 清位与重算 free bits

- 见  storage/innobase/ibuf/ibuf0ibuf.cc

```
4242 reset_bit:
4243     if (update_ibuf_bitmap) {
4244         bitmap_page = ibuf_bitmap_get_map_page(page_id, *page_size,
4245                                                  UT_LOCATION_HERE, &mtr);
4246         ibuf_bitmap_page_set_bits(bitmap_page, page_id, *page_size,
4247                                   IBUF_BITMAP_BUFFERED, false, &mtr);
4248         if (block != nullptr) {
4249             ulint old_bits = ibuf_bitmap_page_get_bits(
4250                 bitmap_page, page_id, *page_size, IBUF_BITMAP_FREE, &mtr);
4251             ulint new_bits = ibuf_index_page_calc_free(block);
4252             if (old_bits != new_bits) {
4253                 ibuf_bitmap_page_set_bits(bitmap_page, page_id, *page_size,
4254                                             IBUF_BITMAP_FREE, new_bits, &mtr);
4255             }
4256         }
4257     }
```

- 页此刻已在手中, 故可将 free bits 更新为真值——这是唯一允许上调 free bits 的时机 (与页修改处于同一 mtr)
- 最后累加统计: n_merges 加一, mops / dops 分别汇入 n_merged_ops / n_discarded_ops (第 4269 行)



主动合并: 后台 master 线程

- 读页触发的合并是**被动的**: 若无人读取该页, 条目将长期留存于 ibuf 树
 - ▶ 故须另设一条**主动**路径, 否则 ibuf 树只增不减
- 入口 `ibuf_merge_in_background(bool full)`, 见 `ibuf/ibuf0ibuf.cc` 第 2398 行
- 三个调用点均位于 `srv/srv0srv.cc` 的 master 线程
 - ① 第 2334 行 `srv_master_do_active_tasks()`: `full=false`, 每秒一次小批量
 - ② 第 2401 行 `srv_master_do_idle_tasks()`: `full=true`, 空闲时全力合并
 - ③ 第 2500 行 `srv_master_do_shutdown_tasks()`: slow shutdown 时合并干净, 前置条件为 `srv_fast_shutdown == 0` 且 `force_recovery` 未禁用 ibuf merge
- 第 3 条即 ibuf 积压较多时 `innodb_fast_shutdown=0` 关机缓慢的原因
- 内部经 `ibuf_merge()` (第 2361 行) → `ibuf_merge_pages()` (第 2231 行):
`pcur.set_random_position()` 随机选取一个叶子页作为起点

合并批量：按 io_capacity 自适应加码

- ibuf_merge_in_background() 函数体，见 [storage/innobase/ibuf/ibuf0ibuf.cc](#)

```
2410 if (full) {
2411     /* Caller has requested a full batch */
2412     n_pages = PCT_IO(100);
2413 } else {
2414     /* By default we do a batch of 5% of the io_capacity */
2415     n_pages = PCT_IO(5);
2416
2417     mutex_enter(&ibuf_mutex);
2418
2419     /* If the ibuf->size is more than half the max_size
2420      then we make more aggressive contraction.
2421      +1 is to avoid division by zero. */
2422     if (ibuf->size > ibuf->max_size / 2) {
2423         ulint diff = ibuf->size - ibuf->max_size / 2;
2424         /* limits to around 100% value, for shrinking max_size case */
2425         diff = std::min(diff, ibuf->max_size);
2426         n_pages += PCT_IO((diff * 100) / (ibuf->max_size + 1));
2427     }
2428
2429     mutex_exit(&ibuf_mutex);
2430 }
```

- PCT_IO(p) 即 innodb_io_capacity 的 p% ([include/srv0srv.h](#) 第 638 行)
- 水位超过 max_size/2 之后，超出越多则追加页数越多，上限约为再加 100%



顺带合并邻页: IBUF_MERGE_AREA

- 既然已为一个页付出一次随机读，便一并将邻近页读入合并

▶ 见 `storage/innobase/ibuf/ibuf0ibuf.cc`

```
295  /** The area in pages from which contract looks for page numbers for merge */
296  const ulint IBUF_MERGE_AREA = 8;
297
298  /** Inside the merge area, pages which have at most 1 per this number less
299  buffered entries compared to maximum volume that can buffered for a single
300  page are merged along with the page whose buffer became full */
301  const ulint IBUF_MERGE_THRESHOLD = 4;
302
303  /** In ibuf_contract at most this number of pages is read to memory in one
304  batch, in order to merge the entries for them in the insert buffer */
305  const ulint IBUF_MAX_N_PAGES_MERGED = IBUF_MERGE_AREA;
```

- `ibuf_get_merge_page_nos()` (第 2005 行) 以 `page_no / IBUF_MERGE_AREA` 划分“合并区”，同区之内最多凑齐 8 个页
- `contract=true` (后台收缩) 时区内所有页照单全收；
`contract=false` (插入容纳不下) 时仅带上缓冲量超过阈值 $3/4$ 的邻页



6

配置、实践与小结



Change Buffer 相关系统变量

变量	默认	范围	定义位置
<code>innodb_change_buffering</code>	<code>all</code>	6 个枚举	☞ ha_innodb.cc 第 22900 行
<code>innodb_change_buffer_max_size</code>	25	0 .. 50	☞ ha_innodb.cc 第 22908 行

- `CHANGE_BUFFER_DEFAULT_SIZE = 25` 定义于 ☞ [include/ibuf0ibuf.h](#) 第 47 行
- 二者均可在线修改
 - ▶ `change_buffer_max_size` 经 `innodb_change_buffer_max_size_update()` (第 20668 行) → `ibuf_max_size_update()` 重算 `ibuf->max_size`
 - ▶ `change_buffering` 直接改写全局 `innodb_change_buffering`, `ibuf_insert()` 每次进入读取一次快照, 以保证单次调用内视图一致
- 启动时由 `innobase_init_files()` 调用 `ibuf_max_size_update(srv_change_buffer_max_size)` (☞ [ha_innodb.cc](#) 第 5581 行)



何时应当关闭

- 收益成立的前提（文档 [Configuring Change Buffering 一节¹](https://dev.mysql.com/doc/refman/8.0/en/innodb-change-buffer.html)）：**I/O 密集 + 二级索引多 + 工作集容纳不进 Buffer Pool**
 - ▶ 典型受益场景：批量导入、日志型写入、宽表多索引
- 下列情形建议调小甚至关闭
 - ① **SSD / NVMe**：随机读代价大幅下降，可省的 I/O 随之减少
 - ② **工作集基本容纳得下 Buffer Pool**：页几乎总在内存，闸门一直接挡回
 - ③ **读多写少**：Change Buffer 徒然占用 Buffer Pool 的额度
 - ④ **需要可预测的关机时间**：ibuf 积压将显著拖长 slow shutdown
- 观测手段
 - ▶ SHOW ENGINE INNODB STATUS 中 Ibuf: size 长期接近 max_size 表明积压
 - ▶ merged operations 长期为 0 而 size 非 0，表明缓冲已成而无人读取
 - ▶ discarded operations 增长表明有表 / 索引被删，条目被白白丢弃
- 折中方案：先将 innodb_change_buffer_max_size 降至 5~10，再考虑 innodb_change_buffering=none

¹<https://dev.mysql.com/doc/refman/8.0/en/innodb-change-buffer.html>



Change Buffer 回顾

- **是什么**：二级索引页不在 Buffer Pool 时，将变更记入一棵全局 B+ 树，待页被读入时再合并回去
- **在哪里**：内存中为 `ibuf_t` 控制结构；磁盘上 header 位于系统表空间 page 3、树根位于 page 4；另有每表空间的 `ibuf bitmap` 页记录“该页有无待合并的变更”
- **什么可缓冲**：`insert / delete mark / delete` 三类，且须通过 `ibuf_should_try()` 的全部条件——核心是非聚簇、非唯一、非降序、非空间索引
- **何时合并**：页被读入时（主路径）、新建页时清理、master 线程后台批量、slow shutdown
- **贯穿始终的不变式**：**merge 必须成功**，由 free bits 的保守低估予以保证
- **设计取向**：**乐观且可退让**——页已在内存不缓冲、记录过大不缓冲、水位过高不缓冲、容纳不下转为主动合并；任何一步退让均不影响正确性
- **下一讲**：Adaptive Hash Index 与 Log Buffer



结束

